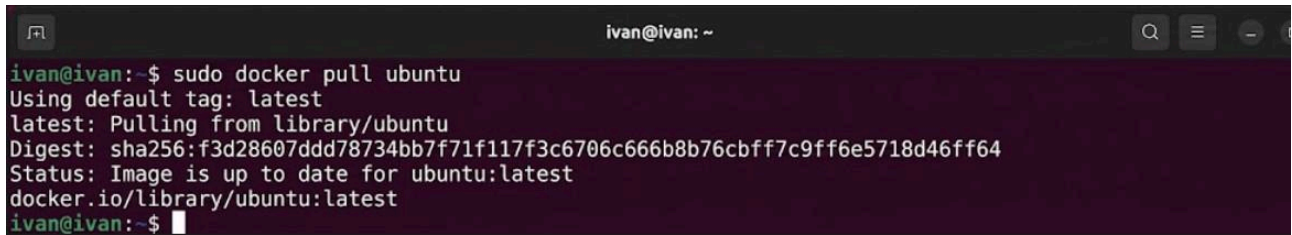


Actividad 3 - Pull, rename y borrado de contenedores

1. Descarga la imagen de Ubuntu

Descargamos la imagen de Ubuntu desde Docker Hub con:

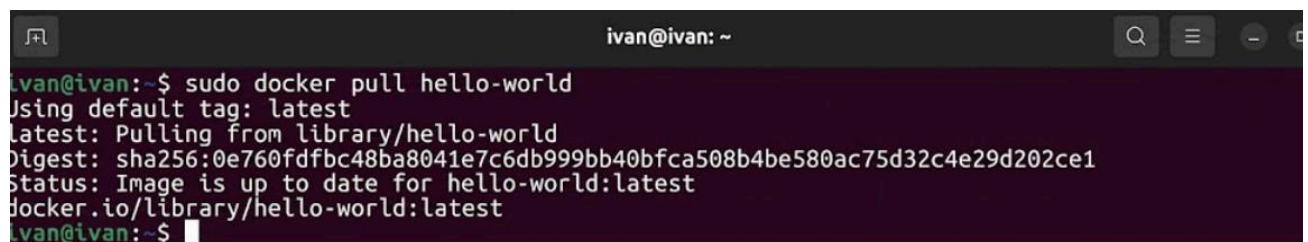
```
sudo docker pull ubuntu
```



```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker pull ubuntu  
Using default tag: latest  
latest: Pulling from library/ubuntu  
Digest: sha256:f3d28607ddd78734bb7f71f117f3c6706c666b8b76cbff7c9ff6e5718d46ff64  
Status: Image is up to date for ubuntu:latest  
docker.io/library/ubuntu:latest  
ivan@ivan:~$
```

2. Descarga la imagen de hello-world

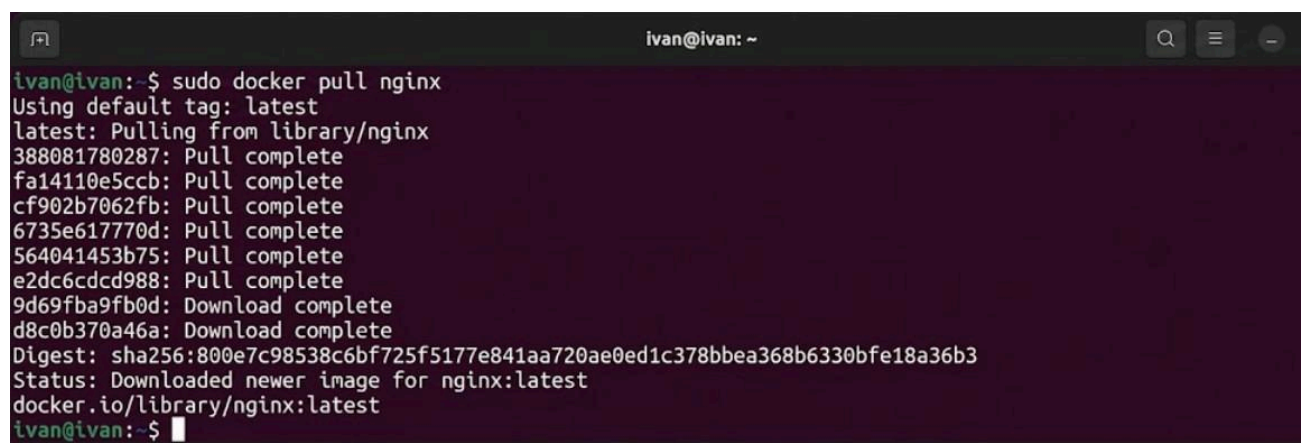
```
sudo docker pull hello-world
```



```
ivan@ivan:~$ sudo docker pull hello-world  
Using default tag: latest  
latest: Pulling from library/hello-world  
Digest: sha256:0e760fd9bc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1  
Status: Image is up to date for hello-world:latest  
docker.io/library/hello-world:latest  
ivan@ivan:~$
```

3. Descarga la imagen de nginx

```
sudo docker pull nginx
```



```
ivan@ivan:~$ sudo docker pull nginx  
Using default tag: latest  
latest: Pulling from library/nginx  
388081780287: Pull complete  
fa14110e5ccb: Pull complete  
cf902b7062fb: Pull complete  
6735e617770d: Pull complete  
564041453b75: Pull complete  
e2dc6cdcd988: Pull complete  
9d69fba9fb0d: Download complete  
d8c0b370a46a: Download complete  
Digest: sha256:800e7c98538c6bf725f5177e841aa720ae0ed1c378bbea368b6330bfe18a36b3  
Status: Downloaded newer image for nginx:latest  
docker.io/library/nginx:latest  
ivan@ivan:~$
```

4. Listado de todas las imágenes

Con docker images vemos todas las imágenes descargadas localmente junto con su tamaño en disco:

```
sudo docker images
```

```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT	EXTRA
hello-world:latest	0e760fdabc48	25.9kB	9.49kB	
httpd:2.4	2a7deaaaf357	177MB	47.6MB	U U
mariadb:latest	350261292d01	468MB	111MB	U U
nginx:latest	800e7c98538c	241MB	66MB	
ubuntu:latest	f3d28607ddd7	160MB	45.3MB	U U

```
ivan@ivan:~$
```

5. Ejecutar contenedor hello-world con nombre myhello1

```
sudo docker run --name myhello1 hello-world
```

```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker run --name my myhello1 hello-world
```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
\$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

```
ivan@ivan:~$
```

6. Ejecutar contenedores myhello2 y myhello3

Repetimos el proceso para crear los contenedores myhello2 y myhello3:

```
sudo docker run --name myhello2  
hello-world sudo docker run --name  
myhello3 hello-world
```

```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker run --name myhello3 hello-world  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/  
ivan@ivan:~$
```

7. Mostrar todos los contenedores

Con `docker ps -a` vemos todos los contenedores, incluidos los parados. Los tres myhello aparecen como Exited porque hello-world termina solo al mostrar su mensaje:

```
sudo docker ps -a
```

```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker ps -a  
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS  
83a7c87409fc   hello-world    "/hello"                19 seconds ago Exited (0) 18 seconds ago  
ebd50dfd8e5e   hello-world    "/hello"                39 seconds ago Exited (0) 38 seconds ago  
8c9c3b64b645   hello-world    "/hello"                About a minute ago Exited (0) About a minute ago  
77c55735ad73   mariadb        "docker-entrypoint.s... 4 minutes ago   Up 4 minutes        3306/tcp  
9efecb62cf24   httpd:2.4      "httpd-foreground"      10 minutes ago Up 10 minutes        0.0.0.0:80->80/tcp  
dffff3f693c1   ubuntu        "bash"                  18 minutes ago Up 17 minutes  
ivan@ivan:~$
```

8. Parar los contenedores myhello1 y myhello2

Aunque ya están detenidos, usamos `docker stop` para practicar el comando:

```
sudo docker stop  
myhello1 sudo docker
```

```
ivan@ivan: ~$ sudo docker stop myhello1
myhello1
ivan@ivan: ~$ sudo docker stop myhello2
myhello2
ivan@ivan: ~$
```

9. Borrar myhello1 y verificar

Borramos myhello1 con docker rm y comprobamos con docker ps -a que ya no aparece, mientras el resto de contenedores siguen ahí:

```
sudo docker rm myhello1
sudo docker ps -a
```

```
ivan@ivan: ~$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
83a7c87409fc	hello-world	"/hello"	3 minutes ago	Exited (0) 58 seconds ago	
ebd50dfd8e5e	hello-world	"/hello"	3 minutes ago	Exited (0) 64 seconds ago	
77c55735ad73	mariadb	"docker-entrypoint.s..."	7 minutes ago	Up 7 minutes	3306/tcp
9efecb62cf24	httpd:2.4	"httpd-foreground"	13 minutes ago	Up 13 minutes	0.0.0.0:8080->80/
dffff3f693c1	ubuntu	"bash"	21 minutes ago	Up 20 minutes	

10. Borrar todos los contenedores

Para borrar todos los contenedores de golpe, combinamos docker ps -aq (que devuelve solo los IDs) con docker rm -f:

```
sudo docker rm -f $(sudo docker ps -aq)
```